

# 基本概念和指令集架构ISA

# 今天的内容

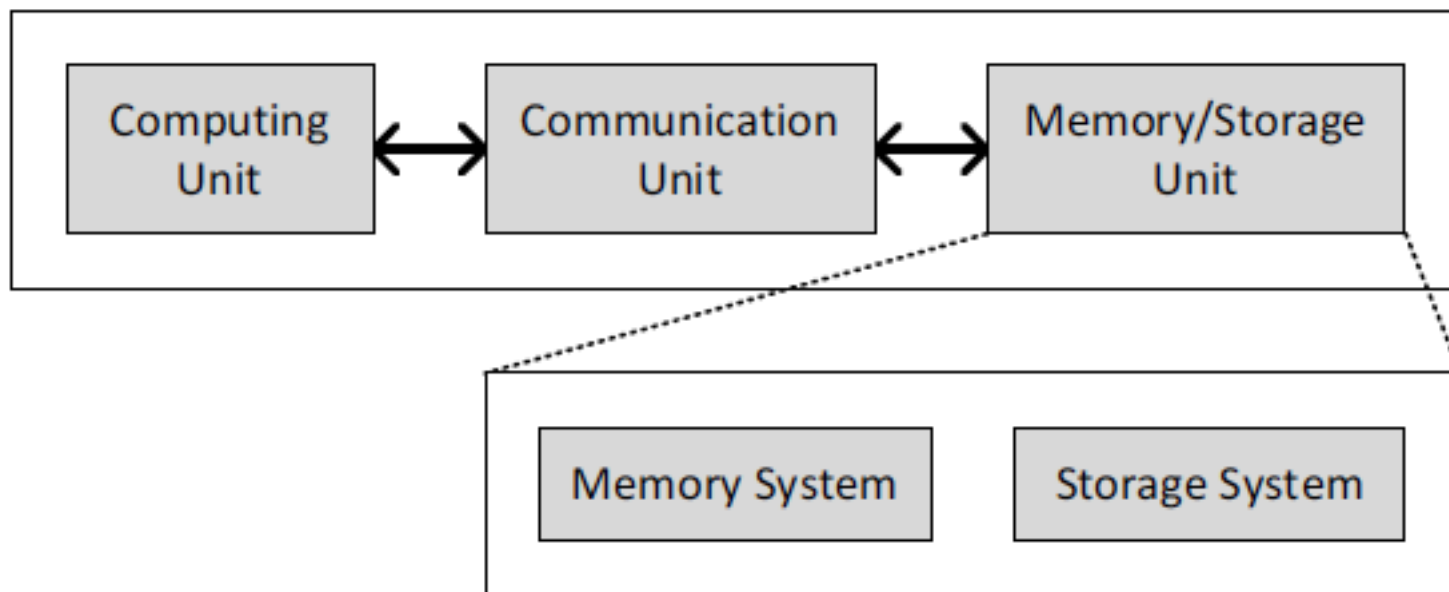
- 基本概念
- 指令集架构ISA

# 什么是计算机？

## ■ 三个核心部件：

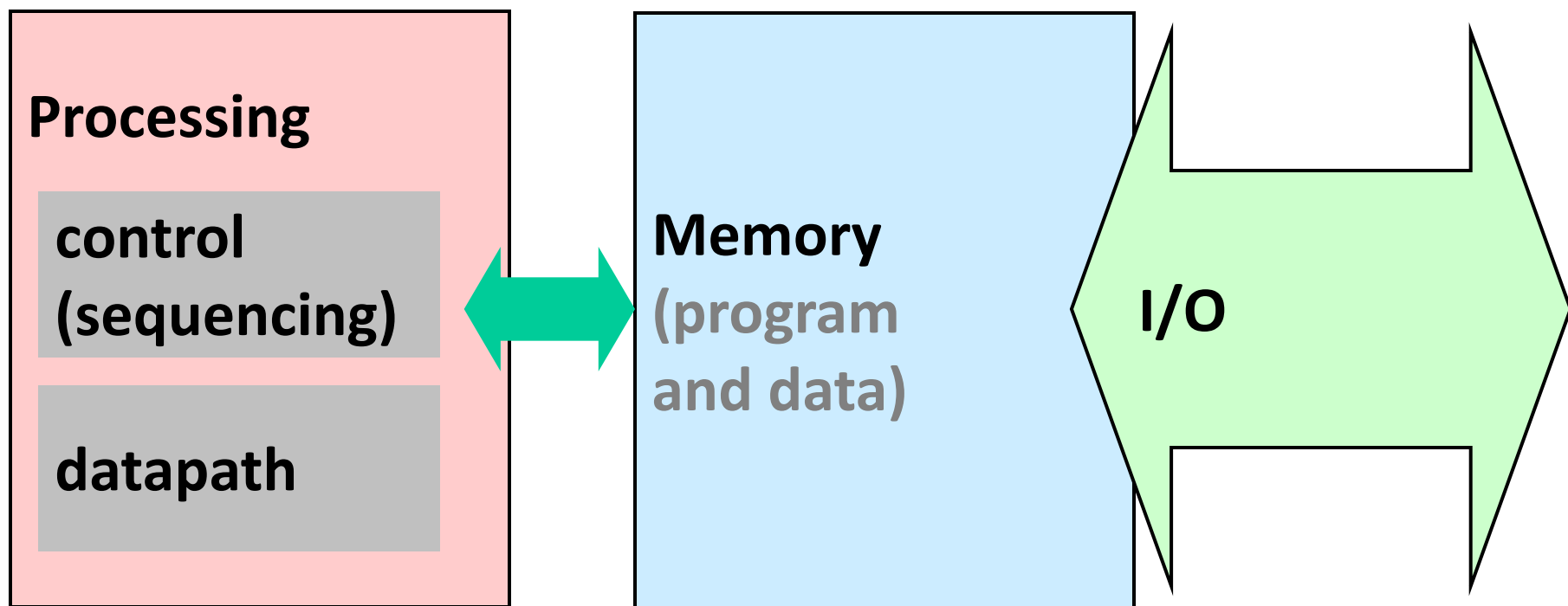
- 计算（Computation）
- 通信（Communication）
- 存储（Storage, memory）

### Computing System



# 什么是计算机？

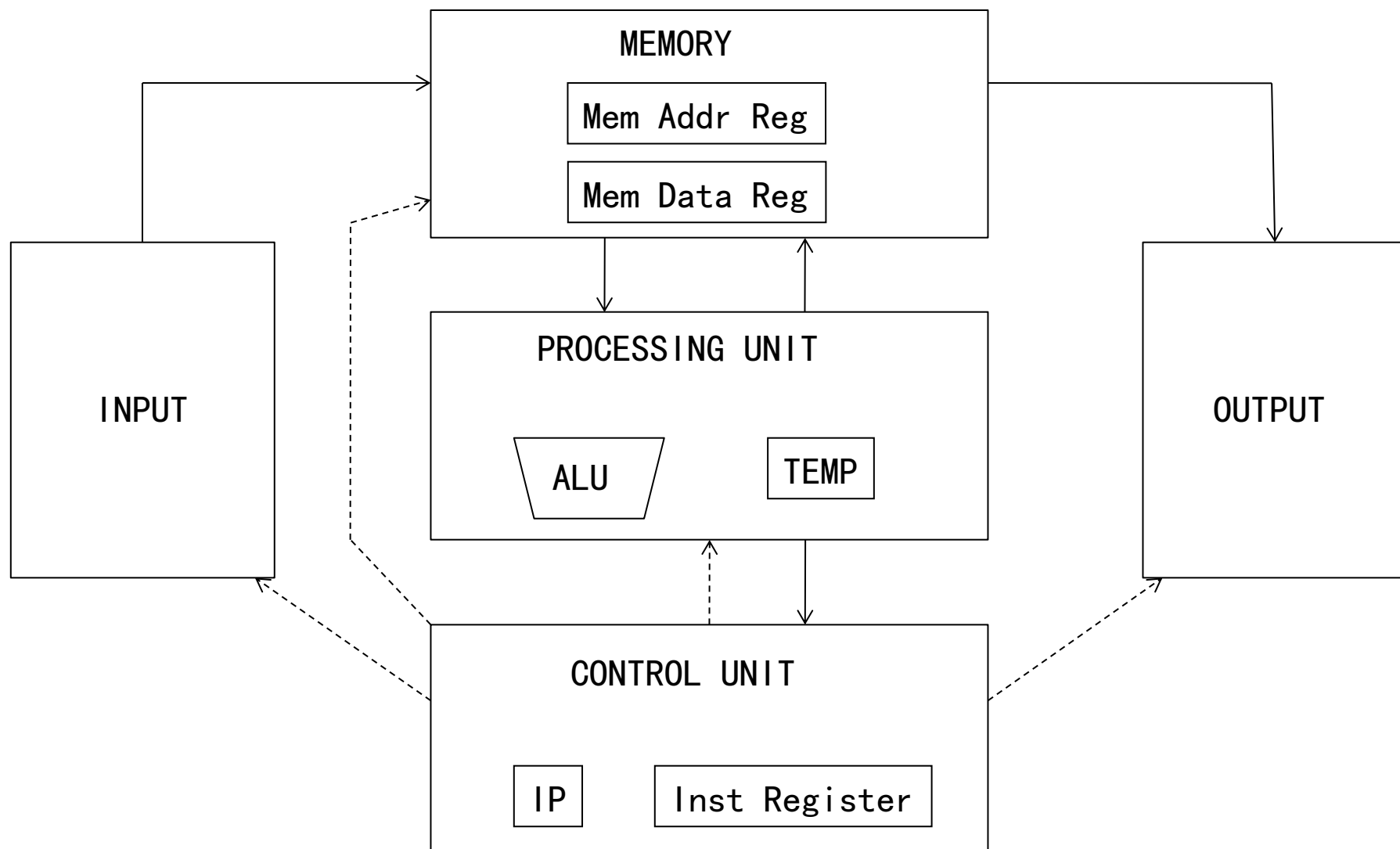
- 覆盖所有三部分



# 冯·诺依曼模型 / 架构

- 也称为存储程序计算机 (stored program computer, 指令存储在内存中)。两个关键特性:
- 存储程序 (Stored Program)
  - 指令以线性方式存储在内存数组中。
  - 指令和数据共用同一存储空间 (Memory is unified between instructions and data)。
  - 存储值的解释取决于控制信号 (The interpretation of a stored value depends on the control signals)。
- 顺序指令处理 (Sequential Instruction Processing)
  - 一次只处理一条指令 (取指、执行、完成)。
  - \*\*程序计数器 (Program Counter, 也叫指令指针)\*\* 标识当前正在执行的指令。
  - 程序计数器按顺序递增, 除非遇到控制转移指令 (如跳转、分支等)。

# 冯·诺依曼模型在计算机中



# 冯·诺依曼模型在计算机中

- 这是计算机运行的唯一方式吗？

- 不是。

- 但是，这一直是主要的方式。

- 即：计算的主流范式

- 已经持续了许多十年

# 数据流模型（计算机的另一种模型）

## ■ 冯·诺依曼模型（Von Neumann model）

- 一条指令的取出和执行遵循控制流顺序（control flow order）
  - 由\*\*指令指针（instruction pointer）\*\*决定
  - 按顺序执行，除非有显式的控制流指令（如跳转、分支）

## ■ 数据流模型（Dataflow model）

- 一条指令的取出和执行遵循数据流顺序（data flow order）
  - 即：当它的操作数（operands）准备好时执行
  - 即：没有指令指针（no instruction pointer）
- 指令执行顺序由\*\*数据流依赖（data flow dependence）\*\*决定
  - 每条指令都会指定“谁”应当接收其结果
  - 当所有操作数到达后，指令即可“触发”（fire）执行
- 潜在地，许多指令可以同时执行
  - 因此天然更具并行性（inherently more parallel）

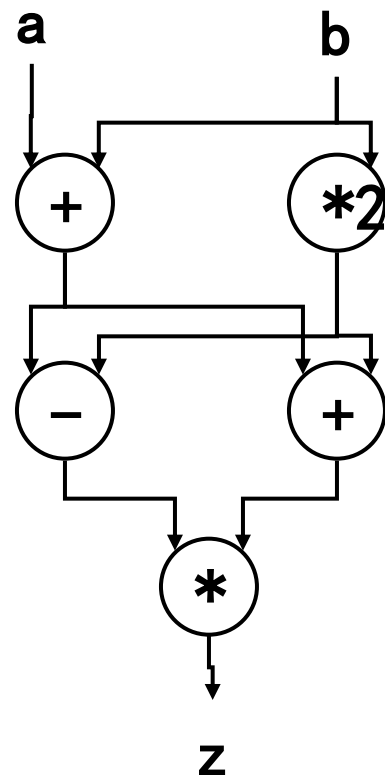
# 冯·诺依曼 vs 数据流

## ■ 思考一个冯·诺依曼程序

- 程序顺序 (program order) 的意义是什么?
- 存储位置 (storage locations) 的意义是什么?

```
v <= a + b;  
w <= b * 2;  
x <= v - w  
y <= v + w  
z <= x * y
```

顺序执行 (Sequential)



数据流执行 (Dataflow)

## ■ 对你作为一名程序员来说, 哪种模型更自然?

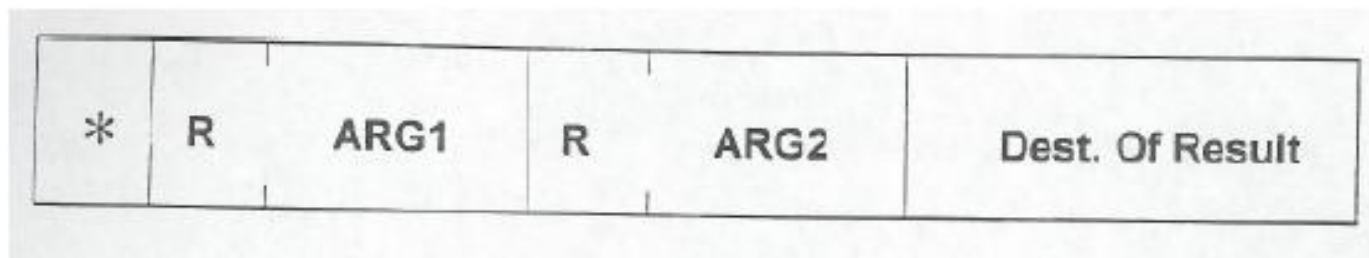
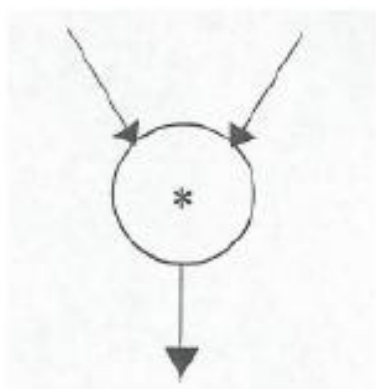
# 数据流模型的进一步说明

## ■ 数据流机器中的程序结构

- 在数据流计算机 (data flow machine) 中, 程序由\*\*数据流节点 (data flow nodes)\*\*组成。
- 当一个节点的所有输入都准备好时, 该节点会触发执行 (fire) (即被取出并执行)。
  - 换句话说: 当所有输入都有\*\*令牌 (tokens)\*\*时, 节点即可执行。

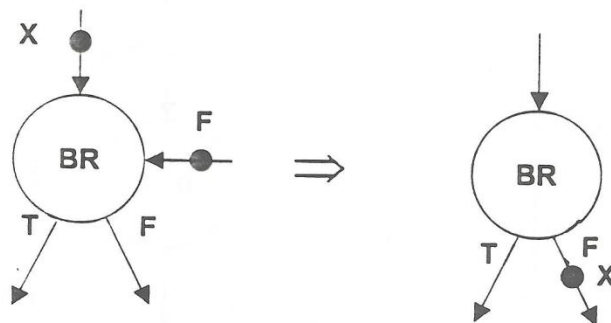
## ■ 数据流节点及其 ISA 表示

- 左图: 一个典型的数据流节点 (例如乘法 \* )。
- 右图: 该节点在指令集架构 (ISA) 中的表示形式。

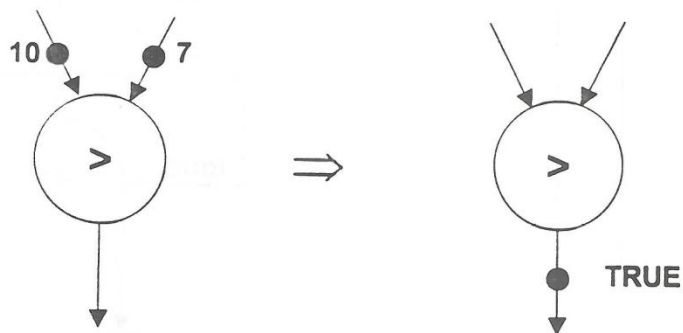


# 数据流节点

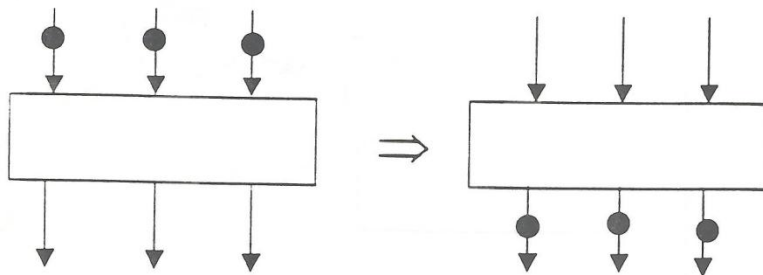
*\*Conditional*



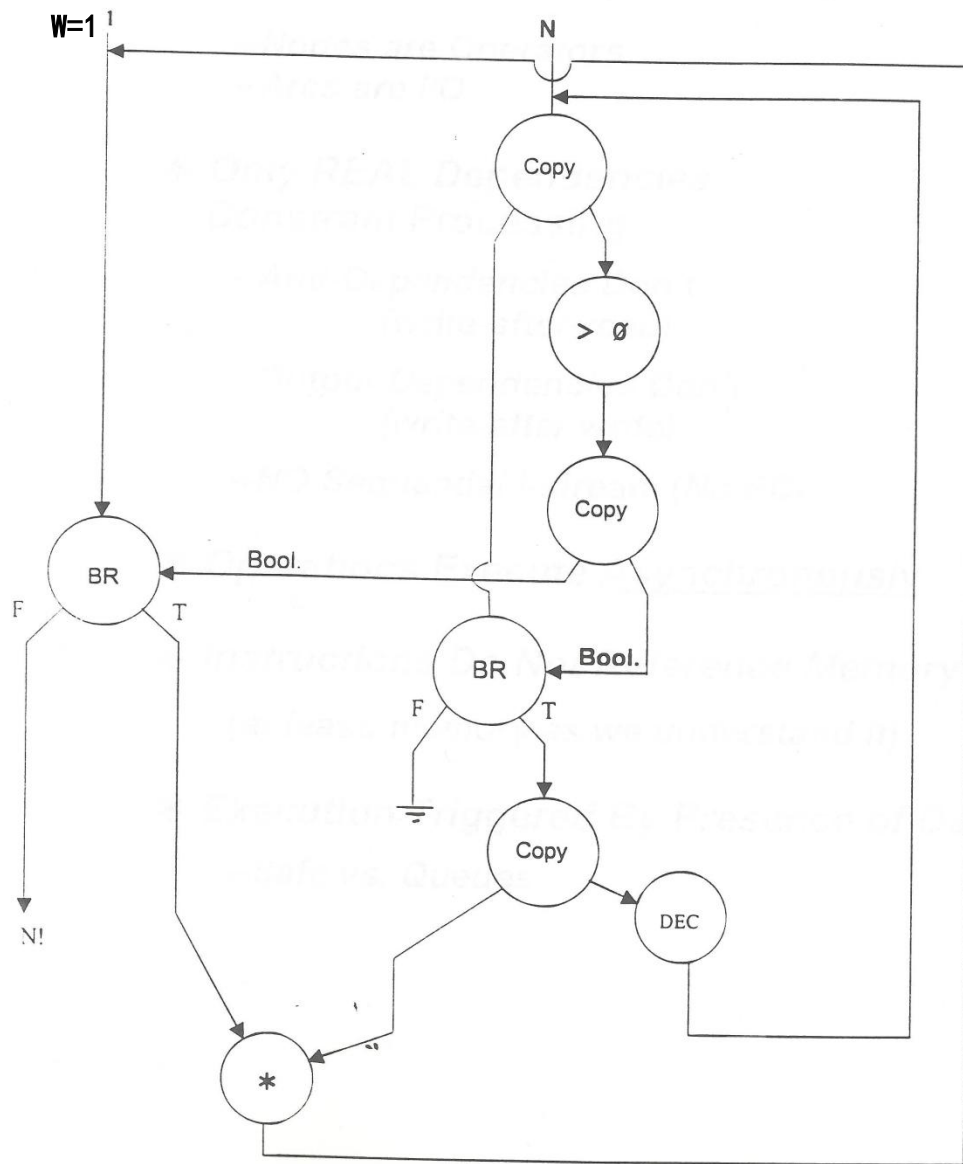
*\*Relational*



*\*Barrier Synch*



# 数据流程序



# 指令集架构（ISA）层面的权衡：指令指针

## ■ 我们是否需要在 ISA 中包含一个指令指针（Instruction Pointer, IP）？

- 需要：控制驱动的顺序执行（Control-driven, sequential execution）
  - 当 IP 指向某条指令时，该指令被执行。
  - IP 会自动按顺序变化（除非遇到控制流指令，例如跳转、分支）。
- 不需要：数据驱动的并行执行（Data-driven, parallel execution）
  - 当指令的所有操作数都可用时，该指令被执行（即数据流执行（data flow））。

## ■ 权衡：许多高层次的取舍（Tradeoffs: MANY high-level ones）

- 编程的容易程度（对于一般程序员而言）？
- 编译的容易程度？
- 性能：并行性的提取（Extraction of parallelism）？
- 硬件复杂度（Hardware complexity）？

# ISA 与微架构层面的权衡

- 类似的权衡（控制驱动 vs 数据驱动执行）也可以发生在微架构层面。
- ISA（指令集架构）：
  - 定义程序员所看到的指令执行方式
  - 程序员可以看到的是顺序、控制流式的执行顺序，或者
  - 程序员可以看到的是数据流式的执行顺序。
- 微架构（Microarchitecture）：
  - 定义底层实现实际上如何执行指令
  - 微架构可以以任何顺序执行指令，只要：
    - 它在向软件暴露执行结果时，遵守 ISA 定义的语义。
  - 换句话说：
    - 程序员应当看到与 ISA 指定顺序一致的结果。

# 回到冯·诺依曼模型

- 但是，如果你想更深入地了解数据流（dataflow）模型……
- 推荐阅读：
  - Dennis and Misunas,  
“*A preliminary architecture for a basic data-flow processor,*”  
ISCA 1974.
  - Gurd et al.,  
“*The Manchester prototype dataflow computer,*”  
CACM 1985.

# 冯·诺依曼模型

- 所有主要的指令集架构（ISA）今天都采用这一模型：x86、ARM、MIPS、SPARC、Alpha、POWER
- 但在底层（微架构层级）几乎所有实现（implementations 或 microarchitectures）的执行模型都与冯·诺依曼模型非常不同：
  - 流水线执行（Pipelined instruction execution）：Intel 80486 架构
  - 同时执行多条指令（Multiple instructions at a time）：Intel Pentium 架构
  - 乱序执行（Out-of-order execution）：Intel Pentium Pro 架构
  - 指令缓存与数据缓存分离（Separate instruction and data caches）
- 但是（But）在底层发生的这些与冯·诺依曼模型不一致的实现细节，不会暴露给软件层。
  - 即：存在ISA 与微架构（microarchitecture）之间的差异。

# ISA 与 微架构 (Microarchitecture)

## ■ ISA (指令集架构)

- 是软件与硬件之间的约定接口 (interface) 。
  - 软件/编译器所假设的，硬件必须保证实现。
- 也是软件开发者编写和调试系统/用户程序所需了解的内容。

## ■ 微架构 (Microarchitecture)

- 是某一 ISA 的具体实现方式 (specific implementation) 。
- 对软件不可见，即程序员无法直接察觉其存在。

|                   |
|-------------------|
| Problem           |
| Algorithm         |
| Program           |
| ISA               |
| Microarchitecture |
| Circuits          |
| Electrons         |

## ■ 微处理器 (Microprocessor)

- 由 ISA、微架构 (uarch) 以及电路 (circuits) 组成。
- 因此：“架构 (Architecture)” = ISA + 微架构 (Microarchitecture)

# ISA 与 微架构

## ■ 什么属于 ISA，什么属于微架构（uarch）？

- 油门踏板（Gas pedal）：是“加速”的接口 → 属于 ISA
- 发动机内部结构（Internals of the engine）：实现“加速”的机制 → 属于 微架构（uarch）

## ■ 实现（uarch）可以多种多样，只要满足规范（ISA）

- “加法指令”（Add instruction）属于 ISA；而“加法器的实现方式”（Adder implementation）属于微架构。
  - 例如：位串行加法器（bit-serial）、连波进位（ripple carry）、超前进位（carry lookahead）加法器 —— 都是不同的微架构实现。
- \*\*x86 指令集（ISA）\*\*有许多不同的实现版本：286、386、486、Pentium、Pentium Pro、Pentium 4、Core 等。

## ■ 微架构通常比 ISA 变化得更快

- 目前存在的 ISA 种类很少（如 x86、ARM、SPARC、MIPS、Alpha），但对应的微架构实现却有很多。
- 思考：为什么？（Why?）

# 指令集架构 (ISA)



## ■ 指令 (Instructions)

- 操作码、寻址方式、数据类型
- 指令类型与格式 (Instruction Types and Formats)
- 寄存器、条件码 (Registers, Condition Codes)

## ■ 内存 (Memory)

- 地址空间、可寻址性、对齐
- 虚拟内存管理 (Virtual memory management)

Intel® 64 and IA-32 Architectures  
Software Developer's Manual

Volume 1:  
Basic Architecture

## ■ 其他内容

- 调用、异常/中断处理 (Call, Interrupt/Exception Handling)
- 访问控制、优先级/特权 (Access Control, Priority/Privilege)
- 输入输出 (I/O) : 内存映射 I/O vs. 指令驱动 I/O
- 任务/线程管理 (Task/thread Management)
- 功耗与热管理 (Power and Thermal Management)
- 多线程与多处理器支持 (Multi-threading and Multiprocessor support)

# 微架构 (Microarchitecture)

- 在特定\*\*设计约束与目标 (design constraints and goals) \*\*下对 ISA 的具体实现。
- 任何在硬件层面完成且不暴露给软件的设计内容都属于微架构范畴。
  - 流水线 (Pipelining)
  - 顺序执行 vs. 乱序执行 (In-order vs. Out-of-order Execution)
  - 内存访问调度策略 (Memory Access Scheduling Policy)
  - 推测执行 (Speculative Execution)
  - 超标量处理 (Superscalar Processing, 多指令并发)
  - 时钟门控 (Clock Gating)
  - 缓存设计 (Caching: 层级、容量、关联度、替换策略)
  - 预取机制 (Prefetching)
  - 电压/频率动态调节 (Voltage/Frequency Scaling)
  - 错误校正 (Error Correction)